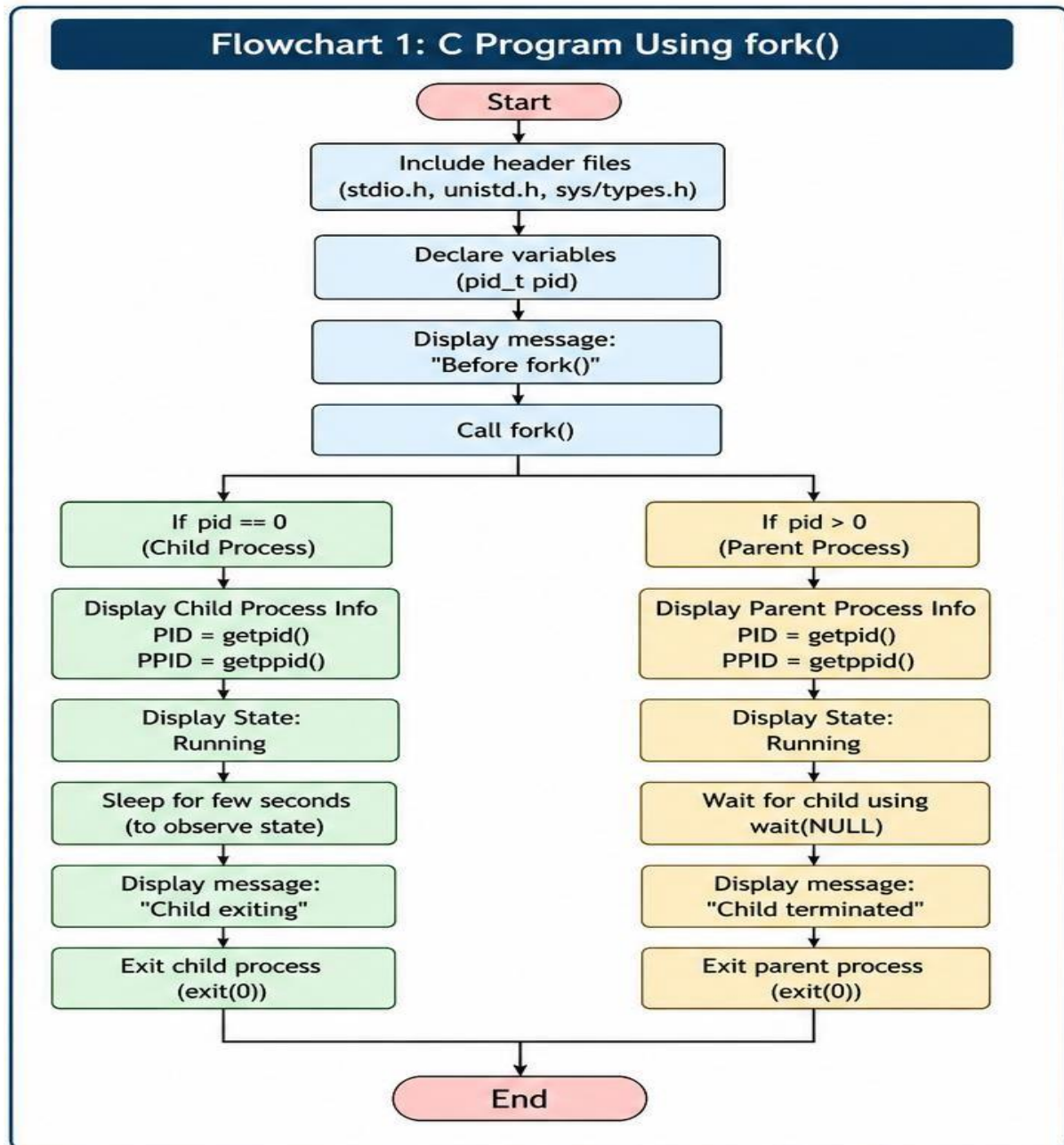


Task 3: Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Aim

To develop a C program using fork() to create a parent and child process and display their PID, PPID and process states.

FlowChart



Program

```
#include <stdio.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>


int main() {

    pid_t pid;


    printf("Before fork():\n");
    printf("PID = %d\n", getpid());
    printf("PPID = %d\n\n", getppid());


    pid = fork();


    if (pid < 0) {
        perror("fork failed");
        return 1;
    }


    if (pid == 0) {
        // Child process
        printf("Child Process:\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        printf("State: Running\n\n");
    }
}
```

```
sleep(5);

printf("Child Process finishing...\n");
}
else {
    // Parent process
    printf("Parent Process:\n");
    printf("PID = %d\n", getpid());
    printf("Child PID = %d\n", pid);
    printf("State: Running\n\n");

    wait(NULL);

    printf("Parent: Child process terminated.\n");
}

return 0;
}
```

Output

```
root@keerthana: ~/week3  X + v
root@keerthana:~# mkdir week3
root@keerthana:~# cd week3
root@keerthana:~/week3# pwd
/root/week3
root@keerthana:~/week3# nano fork_program.c
root@keerthana:~/week3# ls
fork_program.c
root@keerthana:~/week3# gcc fork_program.c -o fork_program
root@keerthana:~/week3# ls
fork_program  fork_program.c
root@keerthana:~/week3# ./fork_program
Before fork():
Process PID  = 534
Parent PPID  = 499

----- PARENT PROCESS -----
Parent PID = 534
Child PID = 535
Parent process is RUNNING
----- CHILD PROCESS -----
Child PID = 535
Child PPID = 534
Child process is RUNNING
Parent is WAITING for child...
Child process is TERMINATED
Child has terminated.
Parent process is TERMINATED
root@keerthana:~/week3# |
```

Observation:

The fork() function successfully created a child process. The parent and child processes had different PIDs, while the child's PPID matched the parent's PID.

Result

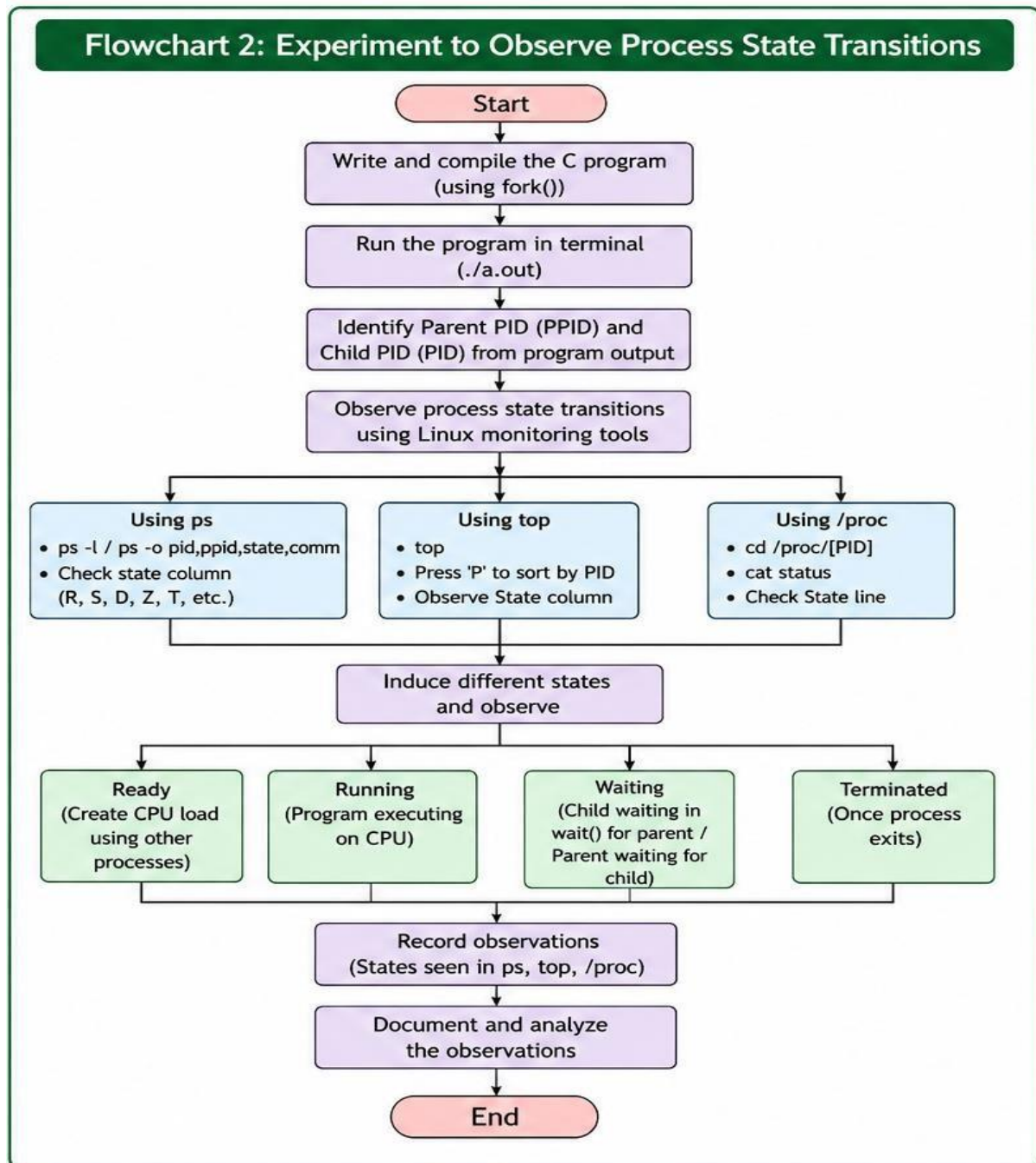
The parent and child processes were successfully created using fork(), and their PID, PPID and states were displayed.

Task 4: Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

Aim

To observe process state transitions using Linux commands such as ps, top and /proc.

Flow Chart:



Program

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main() {
```

```
    printf("Process started!\n");
```

```
    printf("PID = %d\n", getpid());
```

```
    while (1) {
```

```
        sleep(2);
```

```
    }
```

```
    return 0;
```

```
}
```

Procedure

1. Compile and run the program.
2. Note the PID displayed.
3. Use ps to check the process details.
4. Use /proc/<PID>/status to check the process state.
5. Use top to monitor the process.
6. Stop the process using Ctrl+C.

OUTPUT

```
root@keerthana: ~/week3
root@keerthana:~/week3# cd ~/week3
root@keerthana:~/week3# nano process_state.c
root@keerthana:~/week3# gcc process_state.c -o process_state
root@keerthana:~/week3# ./process_state &
[1] 484
root@keerthana:~/week3# Process started!
PID = 484
|
```

```
ps -p 700 -f
UID      PID    PPID  C  STIME TTY          TIME CMD
root@keerthana:~/week3# cat /proc/700/status
cat: /proc/700/status: No such file or directory
root@keerthana:~/week3# ps -p 484 -f
UID      PID    PPID  C  STIME TTY          TIME CMD
root      484     453  0  05:22 pts/0    00:00:00 ./process_st
root@keerthana:~/week3# cat /proc/484/status
Name:   process_state
Umask:  0022
State:  S (sleeping)
Tgid:   484
Ngid:   0
Pid:    484
PPid:   453
TracerPid:      0
Uid:    0       0       0       0
Gid:    0       0       0       0
FDSize: 256
Groups: 0
NSTgid: 484
NSpid:  484
NSpgid: 484
NSsid:  453
Kthread:      0
VmPeak:      2688 kB
VmSize:      2688 kB
VmLck:        0 kB
VmPin:        0 kB
VmHWM:       1672 kB
VmRSS:       1672 kB
RssAnon:           100 kB
RssFile:          1572 kB
```

Observations

1. Using ps

```
root@keerthana: ~/week3
root@keerthana:~# cd ~/week3
root@keerthana:~/week3# nano process_state.c
root@keerthana:~/week3# gcc process_state.c -o process_state
root@keerthana:~/week3# ./process_state &
[1] 484
root@keerthana:~/week3# Process started!
PID = 484
ps -p 700 -f
UID          PID     PPID  C  STIME TTY          TIME CMD
root@keerthana:~/week3# cat /proc/700/status
cat: /proc/700/status: No such file or directory
root@keerthana:~/week3# ps -p 484 -f
UID          PID     PPID  C  STIME TTY          TIME CMD
root         484       453  0   05:22 pts/0      00:00:00 ./process_st
root@keerthana:~/week3# |
```

2. Using /proc

```
ps -p 700 -f
UID          PID     PPID  C  STIME TTY          TIME CMD
root@keerthana:~/week3# cat /proc/700/status
cat: /proc/700/status: No such file or directory
root@keerthana:~/week3# ps -p 484 -f
UID          PID     PPID  C  STIME TTY          TIME CMD
root         484       453  0   05:22 pts/0      00:00:00 ./process_st
root@keerthana:~/week3# cat /proc/484/status
Name:      process_state
Umask:    0022
State:     S (sleeping)
Tgid:     484
Ngid:     0
Pid:      484
PPid:     453
TracerPid: 0
Uid:      0
Gid:      0
FDSize:   256
Groups:   0
NStgid:   484
NSpid:    484
NSpgid:   484
NSSid:    453
Kthreadd: 0
VmPeak:   2688 kB
VmSize:   2688 kB
VmLck:    0 kB
VmPin:    0 kB
VmHWM:    1672 kB
VmRSS:    1672 kB
RssAnon:  100 kB
RssFile:  1572 kB
```

3. Using top

```
root@keerthana: ~/week3
Mems_allowed_list: 0
voluntary_ctxt_switches: 90
nonvoluntary_ctxt_switches: 0
root@keerthana:~/week3# top
top - 05:26:13 up 14 min, 1 user, load average: 0.09, 0.03, 0.01
Tasks: 23 total, 1 running, 22 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 7796.6 total, 7192.3 free, 553.5 used, 200.4 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 7243.1 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
    1 root        20   0   21532  13056  9732  S   0.0   0.2   0:00.48 systemd
    2 root        20   0    3180   2208  2072  S   0.0   0.0   0:00.00 init-systemd(Ub
    6 root        20   0    3196   2124  2008  S   0.0   0.0   0:00.00 init
   52 root        19  -1   34056  12544  11464  S   0.0   0.2   0:00.14 systemd-journal
  101 root        20   0   25284   6692   5180  S   0.0   0.1   0:00.32 systemd-udev
  111 systemd+    20   0   21344  13124  11000  S   0.0   0.2   0:00.09 systemd-resolve
  112 systemd+    20   0   91036   7980   7012  S   0.0   0.1   0:00.05 systemd-timesyn
  172 root        20   0    4244   2684   2432  S   0.0   0.0   0:00.00 cron
  173 message+    20   0    9544   5304   4612  S   0.0   0.1   0:00.04 dbus-daemon
  181 root        20   0   17980   8888   7848  S   0.0   0.1   0:00.07 systemd-logind
  183 root        20   0 1756104  13192  10884  S   0.0   0.2   0:00.09 wsl-pro-service
  190 syslog      20   0  222516    6012   4652  S   0.0   0.1   0:00.06 rsyslogd
  203 root        20   0    3124   1972   1836  S   0.0   0.0   0:00.00 agetty
  209 root        20   0 107032  23180  13724  S   0.0   0.3   0:00.11 unattended-upgr
  297 root        20   0    6672   4572   3804  S   0.0   0.1   0:00.00 login
  340 root        20   0   20120  11316   9380  S   0.0   0.1   0:00.06 systemd
  341 root        20   0   21172    3520   1804  S   0.0   0.0   0:00.00 (sd-pam)
  366 root        20   0    6080   5340   3700  S   0.0   0.1   0:00.00 bash
  449 root        20   0    3184   1120    980  S   0.0   0.0   0:00.00 SessionLeader
  451 root        20   0    3200   1256   1108  S   0.0   0.0   0:00.12 Relay(453)
  453 root        20   0    6080   5380   3676  S   0.0   0.1   0:00.02 bash
  484 root        20   0    2688   1672   1572  S   0.0   0.0   0:00.00 process_state
```

4. After Termination

```
root@keerthana:~/week3# kill 484
root@keerthana:~/week3# ps -p 484
  PID TTY          TIME CMD
[1]+  Terminated                  ./process_state
root@keerthana:~/week3# |
```

Result

The process was successfully created, monitored using ps, top and /proc, and finally terminated. The required process states were observed during the experiment.